MAVAI +
DR MANATHAN

Audit, cahier des charges et roadmap

Version 1.0 · 14 novembre 2026 · Europe/Paris

Dossier opérationnel pour l’agent Lovable

Socle MAVAI conservé. Mode généraliste et nouvelles intégrations à construire par tranches vérifiables.



QR vers la plateforme actuelle : https://mavai-studio.lovable.app

À remplacer pour la nouvelle instance après publication. Scan téléphone non effectué ici.

1. Décision, périmètre et état réel

MAVAI peut accueillir DR MANATHAN par ajout de capacités. Le socle actuel possède déjà identité, projets, conversations, mémoire, production audiovisuelle et plusieurs fournisseurs de modèles. Le travail prioritaire est de réunir un mode généraliste, des outils exécutables, des commandes vocales et des preuves de résultat dans cette architecture.

Réalisé pendant cette mission	État
Accès et sources	Connexion Doppler confirmée ; inventaire de 10 projets et 43 configurations, noms seulement. Dépôts MAVAI et bnb lus. Documentation officielle comparée.
Code ajouté	SDK Higgsfield TypeScript et exemple serveur isolé, gestion des résultats et tests. Aucun changement aux écrans, schémas ou fonctionnalités de production.
Contrôles	Build réussi ; 79 tests MAVAI et 17 tests Higgsfield réussis ; typecheck et lint ciblés réussis. 16 tests SPRITES 2D réussissent avec une configuration de contrôle dédiée.
OpenAI actualisé	OPENAI_API_KEY_2 : catalogue et génération gpt-5.4 HTTP 200 ; réponse attendue ; 28 tokens au total. Les 50 USD de crédit sont déclarés par le demandeur, sans vérification du solde.
Blocages constatés	Higgsfield : une tentative, refus HTTP 403 du SDK ; aucune vidéo. Nouvelle clé ElevenLabs : compte accessible, TTS HTTP 401, STT non atteint. Oriane : authentification non décrite dans l'OpenAPI public.
À réaliser par Lovable	Copie du projet dans un nouvel environnement, mode DR MANATHAN, commandes audio/texte, connecteurs métier, QR sur accueil, recette de bout en bout et publication du clone.

Référence MAVAI : d82ccc8091a40cbc57f67cb85a292f21928cbef1. Référence bnb : da5e2cf9e6dc8399f357d36ac05df7c89ec61583. L'application actuelle déclarée par le dépôt est https://mavai-studio.lovable.app (HTTP 200 observé). Aucun nouvel environnement Lovable, domaine ou compte de trading n'a été créé pendant cette mission. Les ajouts sont livrés sur une branche dédiée ; la branche principale reste conservée.

Événement fourni par le demandeur : hackathon Open Data swanbase / Hémicycle / Featherless, samedi 14 novembre 2026, 10 h-19 h à Paris. Objectif de démo : une question ouverte en français, une recherche Hémicycle vérifiable, un résultat sourcé et l'accès mobile par QR. Les crédits ou performances annoncés par l'organisateur restent des informations événementielles tant qu'ils ne sont pas vérifiés sur les comptes.

2. Technologies : utilité et décision

Technologie	Rôle dans MAVAI	Décision pour DR MANATHAN
TypeScript	Contrats de données et code partagé interface/serveur.	Conserver ; exemples et adaptateurs dans la langue du projet.
React 19	Interface, composants et état de session.	Ajouter les contrôles de mode au chat existant.
TanStack Start / Router	SSR, routes et fonctions serveur authentifiées.	Garder /chat et les fonctions serveur comme point d'entrée.
Vite 8 / Nitro / config Lovable	Compilation et packaging du déploiement.	Conserver la configuration intégrée et les frontières serveur.
Bun / bun.lock / workspaces	Installation reproductible et cinq packages SPRITES 2D.	Utiliser Bun ; garder la garde de publication de 24 h du bunfig.
AI SDK / openai-compatible	Streaming et transport de réponses vers le chat.	Ajouter les outils typés et vérifier les capacités par modèle.
Lovable AI gateway	Réponses intégrées, images, musique et recherche de repli.	Préserver les appels existants et la clé serveur.
Supabase / Lovable Cloud	Authentification, données et contrôles de propriétaire.	Réutiliser ; préparer les changements de schéma dans Lovable.
Drizzle	Migrations du projet et typages générés.	AGENTS interdit de modifier les migrations à la main.
Zod	Validation des entrées et identifiants.	Décrire chaque outil et ses paramètres avec un schéma strict.
Tailwind 4 / Radix / Lucide	Styles, accessibilité, interactions et icônes.	Garder thèmes/tokens et surfaces existantes.
Streamdown / Markdown / Mermaid	Rendu des réponses, code et diagrammes.	Réutiliser pour les résultats généralistes et leurs sources.
Web Audio / MediaRecorder	Capture micro, analyse et lecture dans le navigateur.	Séparer entrée micro, sortie parlée et arrêt de traitement.
Canvas / IndexedDB / WAV	Frames et médias locaux, exports et conservation locale.	Conserver originaux et préciser la portée locale des fichiers.
AES-256-GCM / node:crypto	Chiffrement serveur des clés personnelles.	Conserver ; gérer explicitement la clé de chiffrement lors de la copie.
GitHub / gateway connecteur	Mémoire personnelle dans un dépôt privé vérifié.	Maintenir vérification owner/repo et reconfigurer la connexion du clone.
Vitest / ESLint / TypeScript	Vérifications de comportement et statiques.	Étendre la recette sans remplacer la base existante.
SPRITES 2D	Assemblage déterministe, schémas, qualité et SDK.	Préserver les cinq packages et leurs tests séparés.
Nouveaux services	Hémicycle : données ; Featherless : modèles ; Higgsfield : vidéo ; Oriane : veille sociale.	Adaptateurs serveur séparés, statuts et coûts distincts.

Preuves principales : package.json, bunfig.toml, AGENTS.md ; src/lib/chat-stream.server.ts ; src/lib/mavai-ai.server.ts ; src/lib/connectionKeyCrypto.server.ts ; packages/sprites2d-*. Les versions ci-dessus décrivent le socle du dépôt, pas une prescription de mise à niveau générale.

3. Capacités existantes et écarts à combler

Domaine	Implémentation existante	Ajout attendu
Chat	Streaming, historique reconstruit au serveur, hub de projets, conversations et images.	Profil généraliste, outils métier et documents PDF/CSV.
Modèles	OpenAI, Claude, Gemini, Mistral et Perplexity ; dernière connexion de chat vérifiée.	Matrice de capacités, Featherless et tests outils/streaming par modèle.
Mémoire	Projets, mémoire personnelle, liens de chats et mémoire GitHub.	Recherche documentaire pertinente, citations et limites de contexte explicites.
Voix	STT/TTS ElevenLabs et file FIFO commune au texte et à la voix.	Reconnaître et exécuter les commandes ; séparer silence et micro.
Production	Ressources, personnages, versions, storyboard, continuité, image, musique, audio et frames.	Conserver ces modules, leur navigation et leurs données.
Agents	Quatre étapes LLM séquentielles.	Registre d'outils et jobs durables avec résultats observés.
Recherche	Google News RSS pour la disponibilité web et recherche au repli Lovable.	Connecteurs sourcés et recherche métier Hémicycle.
Diagnostic	Voyants catalogue modèle, web, ARTcB et mémoire GitHub.	Santé par capacité : auth, texte, audio, outils et quotas.
BNB	Vidéos de présentation et guide externe.	Cadrage du produit, données de marché et simulation ; moteur à construire.

Points de code utiles : prompt audiovisuel dans chat-stream.server.ts:92 ; choix des fournisseurs dans user-model.server.ts:4 ; file et capture dans voice-session.ts:84-182 ; contrôles dans StreamingChat.tsx:118-156 ; audio serveur dans mavai-ai.server.ts:105-153 ; accueil dans routes/index.tsx:129. Les tests statiques de ce dossier ne certifient pas les flux de production authentifiés.

4. Références des assistants généralistes

Comparaison ciblée d'OpenAI, Anthropic, Mistral et Google sur leurs documentations officielles. Les fonctions d'une application grand public et les capacités de son API ont des conditions d'accès distinctes. MAVAI doit implémenter son produit autour des APIs réellement disponibles.

Plateforme	Capacités utiles documentées	Application à MAVAI
OpenAI / ChatGPT	Outils, recherche, fichiers, code ; projets, voix et tâches dans le produit.	Réutiliser le fournisseur existant ; compléter les outils et leur exécution. [S4]
Anthropic / Claude	Outils exécutés par le client ou le fournisseur, MCP, mémoire avec stockage applicatif.	Traiter tool_use/tool_result et garder la mémoire sous contrôle de MAVAI. [S5]
Mistral	Function calling, connecteurs, recherche, documents, code et voix selon API.	Le Chat Completions ne donne pas automatiquement web_search/code_interpreter ; examiner Agents SDK pour un nouveau besoin. [S6]
Google / Gemini	Search, fichiers, code, outils personnalisés et Live selon modèle.	Tester les combinaisons exactes ; ajouter les capacités au registre par modèle. [S7]

Ordre de valeur recommandé : conversation fiable > recherche avec sources > lecture de documents > outils autorisés > exports > tâches durables > environnement de code isolé. Une infrastructure multi-agents distribuée peut attendre que les usages et contraintes la justifient.

5. Architecture cible et contrats

• Un seul chat et deux profils : audiovisual par défaut pour les anciennes conversations, dr_manathan sur demande. Le profil choisit la politique et les outils accessibles ; les projets et historiques restent conservés.

• Un routeur partagé reçoit texte ou transcription, identifie les commandes déterministes puis transmet les questions au modèle. Les données d'un document ou d'un résultat web restent des contenus à analyser.

• Un registre serveur expose nom, version, schéma Zod, permission, portée owner/project, coût indicatif, délai, annulation et politique de reprise pour chaque outil.

• La politique serveur vérifie la session, la propriété, les limites et l'idempotence avant exécution. Le modèle reçoit des résultats expurgés et des sources, jamais les credentials.

• Les actions courtes restent dans les fonctions serveur ; les générations longues et automations passent par un worker persistant avec état de job et reprise après rechargement.

• Le rendu présente séparément la demande, son accusé, son exécution et son résultat. Une réponse du modèle ne constitue pas à elle seule une action exécutée.

Contrat	Champs et invariants proposés
CommandEnvelope	command_id UUID, source text|voice, action, args validés, conversation_id, project_id, mode, created_at. owner_id vient de la session serveur.
ToolDefinition	id/version, inputSchema, outputSchema, capabilities, permission, budget, timeout, cancel support, retry policy.
ActionEvent	command_id, sequence, state, safe_summary, timestamp. recognized > acknowledged > executing > succeeded|failed|canceled ; unknown reste explicite si effet distant incertain.
ToolResult	Statut, données validées, source_url, retrieved_at, période/version de données, licence si fournie, limites, coût/usage et identifiant de requête.
CapabilityHealth	Provider + modèle + capacité + date + test effectué + verdict. Un catalogue 200 ne vaut pas une génération réussie.

Schéma logique à faire générer par Lovable : mode de conversation ; action_jobs avec unicité owner_id/project_id/idempotency_key ; action_events ordonnés ; documents/chunks/sources isolés par propriétaire ; configuration de capacités et diagnostics. Réutiliser les tables et types existants quand leur rôle convient. Les migrations et fichiers générés restent gérés par Lovable conformément à AGENTS.md.

Points d'extension proposés : answer-policy.ts pour les règles communes ; chat-stream.server.ts et chat.functions.ts pour le profil partagé ; StreamingChat.tsx et voice-session.ts pour les commandes ; nouveaux modules command-router, tool-registry.server, adapters/hemicycle.server et adapters/featherless.server ; routes/index.tsx pour le QR. Ces noms nouveaux sont des propositions de conception.

6. Cahier des charges des commandes vocales

Les mêmes commandes doivent fonctionner au clavier et après transcription. L'accusé configurable « Oui chef, … » est visible avant le dispatch. Il est audible si la sortie parlée est autorisée. Il reste une préférence personnelle ; la liste d'expertises demandée par cet utilisateur suit la même règle de personnalisation.

Ordre / identifiant proposé	Comportement vérifiable
« Passe en mode DR MANATHAN » / mode.set	Accusé ; mode généraliste actif et badge visible ; conversation et projet conservés.
« Reviens à MAVAI » / mode.set	Retour au profil audiovisuel, sans supprimer de ressources.
« Mode silence » / speech.disable	Accusé court puis arrêt de la lecture en cours et des prochains TTS. Le micro conserve son état.

Ordre / identifiant proposé	Comportement vérifiable
« Réactive la voix » / speech.enable	Accusé texte avant activation ; réponses suivantes audibles si le navigateur autorise la lecture.
« Coupe le micro » / microphone.disable	Micro désactivé et état affiché ; la reprise exige un clic si aucune autre entrée n'est active.
« Stop » / turn.cancel	Accusé texte immédiat puis arrêt prioritaire local, hors attente FIFO. Une annulation distante attend sa confirmation ; état inconnu si nécessaire.
« Répète la réponse » / answer.replay	Relit la réponse retenue sans refaire une génération ou une action facturable.
« Nouvelle conversation dans … » / conversation.create	Vérifie le projet détenu, crée un enregistrement et y navigue après l'accusé.
« Ouvre mon projet … » / project.open	Résout un identifiant détenu ; clarification si plusieurs noms correspondent.
« Recherche dans Hémicycle … » / data.search	Appel de l'adaptateur autorisé ; résultats, sources et date visibles.
« Montre les sources » / sources.show	Affiche les sources réellement utilisées et leurs dates.
« Résume ce document » / document.summarize	Utilise seulement le document choisi et détenu ; référence les passages pertinents.
« Exporte en CSV/PDF » / artifact.export	Produit un fichier et fournit un lien ; aucune annonce de succès sans fichier créé.
« Lance/arrête la simulation BNB » / trading.simulation	Agit sur un compte simulé identifié et affiche son état ; les ordres réels suivent un mandat distinct.

• Les phrases citées (« écris le mot silence »), les paroles d'un média, une réponse d'outil ou une reconnaissance ambiguë ne doivent pas exécuter un ordre.

• Chaque commande possède un identifiant stable. Une transcription répétée ou un retry réseau ne doit pas créer une seconde action.

• Pour silence pendant la lecture : publier l'accusé visible immédiatement, puis interrompre la sortie actuelle et appliquer le nouvel état. Ne pas attendre un TTS réseau pour un arrêt urgent.

• Les questions ordinaires conservent la FIFO et la propriété de lecture de chaque tour. Le comportement des tours ultérieurs après Stop doit être défini dans l'interface.

• Un micro coupé ne peut pas entendre « réactive le micro ». Une reprise vocale n'est proposée que si une écoute consentie reste active.

• La détection actuelle segmente après environ 1,2 s de silence puis utilise un STT distant. Une interruption rapide exige une voie de contrôle dédiée ; fixer et mesurer son délai, sans promettre un arrêt vocal instantané avec le pipeline actuel.

Objectifs de recette proposés : accusé textuel affiché en moins de 300 ms après reconnaissance locale de la commande ; arrêt du lecteur en moins de 250 ms après reconnaissance ; mesurer séparément le délai parole > transcription. Ces seuils sont des objectifs à mesurer, pas des performances constatées.

7. API : preuves, disponibilité et blocages

Les contrôles ont utilisé des processus serveur alimentés par Doppler. Les sorties contiennent des statuts, des noms de variables et des compteurs. Les valeurs des credentials n'ont été ni affichées dans le chat, ni intégrées au dépôt ou aux livrables. Projet de secrets pertinent : lvx, configuration dev pour les tests.

Service	Preuve obtenue	Suite nécessaire
OpenAI nouveau	OPENAI_API_KEY_2 : /models 200 ; gpt-5.4 /chat/completions 200, réponse OK ; 12 tokens entrée + 16 sortie.	Configurer cette clé dans le clone via la voie serveur/propriétaire ; tester ensuite streaming, vision et outils.
OpenAI anciennes entrées	OPENAI_API_KEY : 401. OPENAI_API_KEY2 : /models 200.	Conserver les noms distincts et choisir explicitement la nouvelle source ; éviter un changement global silencieux.
Mistral	MISTRAL_API_KEY : /v1/models 200, contrôle sans clé 401 ; génération mistral-large-latest refusée HTTP 403.	Vérifier accès/autorisation de génération ; outils et voix à tester séparément.
Featherless	FEATHERLESS_AI_API : /v1/plan 200 ; génération Qwen/Qwen3-VL-4B-Instruct 200, réponse OK, 12 tokens.	Texte vérifié ; tester streaming, outils et vision avant de les activer.
Hémicycle	HEMICYCLE_AI_API : initialize et tools/list 200 ; 18 outils ; appel gratuit lister_jeux 200 non vide.	Lecture MCP vérifiée ; valider chaque outil métier utilisé et son coût.
ElevenLabs	ELEVENLABS_API_KEY et MAVAI_DR_MANATHAN : compte /v1/user 200. Nouveau TTS : 401 ; STT non atteint.	Corriger accès/scope de la clé ou voix, puis tester TTS+STT. La cause exacte du 401 reste à établir.
Higgsfield	HIGGSFIELD_AI_API a le format attendu ; mapping runtime HF_CREDENTIALS. subscribe refusé HTTP 403 / NotEnoughCreditsError.	Vérifier crédits et accès modèle côté compte ; ne relancer qu'après correction. Aucun lien vidéo produit.
Oriane	Documentation /rest/docs et free-tools accessibles. Auth absente de l'OpenAPI public.	Obtenir le contrat d'auth ; aucun header supposé et aucun appel authentifié lancé.
ARTcB	Noms ARTCB_API_URL/KEY présents ; URL du probe ne passe pas le contrôle HTTPS.	Corriger la configuration de cet environnement et tester health puis fonction métier séparément.
Supabase / Lovable	SUPABASE_URL et alias PUBLISHABLE_KEY_SUPABASE présents. /auth/v1/health échoue côté transport/client, sans statut HTTP.	Réconcilier les noms et vérifier backend, auth et clé de chiffrement du clone ; fonctionnement non confirmé ici.
Autres fournisseurs existants	Claude/Gemini/Perplexity/Replicate et connecteur mémoire présents dans le code.	Tests de compte et capacités non réalisés pour tous ; compléter avec les credentials du clone.

Absences sous leur nom exact dans les configurations lvx inspectées : LOVABLE_API_KEY, APP_USER_CONNECTION_KEY_SECRET, SUPABASE_PUBLISHABLE_KEY et les variables VITE Supabase attendues. Leur présence dans Lovable Cloud peut être gérée indépendamment de Doppler. Le nom PUBLISHABLE_KEY_SUPABASE est un alias candidat à vérifier. Une API_KEY_SUPABASE ne doit pas être assimilée à une service-role sans vérification. Ne pas recopier la clé de chiffrement d'une autre application au hasard.

Le chemin vocal du code lit ELEVENLABS_API_KEY serveur ; une entrée MAVAI_DR_MANATHAN dans Doppler ou une clé personnelle sauvegardée dans Paramètres ne l'alimente pas automatiquement. Faire un mapping explicite dans le runtime du clone après test fonctionnel.

Hémicycle utilise MCP Streamable HTTP avec Authorization: Bearer et l'URL https://api.hemicycle.ai/mcp. Les appels guide/lister_jeux/solde_credits sont documentés gratuits ; les autres coûts dépendent de l'outil. Une erreur GET 405 sur /mcp n'est pas une invalidation de clé. Featherless utilise une base compatible OpenAI https://api.featherless.ai/v1. [S8, S9]

Oriane complète la veille vidéo et sociale. Les routes publiées concernent profils, contenus et création d'assets ; l'authentification reste à documenter. Ces données, comme les données parlementaires, ne remplacent pas un flux de prix et de liquidité pour le trading. [S10]

8. Exigences fonctionnelles et critères de réception

ID / priorité	Exigence	Preuve de réception attendue
F01 / P0	Copie isolée de MAVAI et référence source conservée.	Nouvelle identité Lovable/backend ; comparaison des fonctionnalités ; ancien projet accessible.
F02 / P0	Deux modes dans /chat ; historique conservé.	Ancien chat audiovisuel inchangé ; bascule DR et retour après reload.
F03 / P0	Routeur unique texte/audio et accusé avant action.	Même enveloppe et même effet via texte et transcription ; événement acknowledged antérieur à executing.
F04 / P0	Silence, reprise, micro et arrêt distincts.	Test lecture en cours, file chargée, micro coupé, reprise et autoplay bloqué.
F05 / P0	Hémicycle en lecture avec sources.	Une question réelle, paramètres conservés, résultat et citation vérifiables.
F06 / P0	QR accueil de la nouvelle instance.	Lien HTTPS sans jeton ; scan téléphone depuis écran de présentation jusqu'à auth/démo autorisée.
F07 / P0	Diagnostic honnête des APIs.	HTTP/erreur/capacité manquante affichés ; aucun voyant « prêt » fondé uniquement sur présence de clé.
F08 / P1	Featherless intégré au registre des fournisseurs.	Texte et streaming testés ; capacités outils/vision actives seulement après tests dédiés.
F09 / P1	Recherche et réponses sourcées.	Liens, dates et limites visibles ; chiffres reproduits depuis la donnée exacte.
F10 / P1	Lecture PDF/CSV et mémoire documentaire.	Extraction serveur, citations de passages, suppression/rétention et isolation entre projets.
F11 / P1	Exports de fichiers.	Téléchargements PDF/CSV vérifiés ; statut produit après création effective.
F12 / P1	Exemple Seedance et intégration média par jobs.	SDK serveur, statut terminal ; vidéo réelle accessible après correction du compte.
F13 / P2	Tâches longues et récurrentes.	Job survit à reload ; annulation et reprise sans répéter une mutation ou une facturation.
F14 / P2	Analyse de données/code dans une sandbox.	Temps, mémoire, réseau et fichiers bornés ; aucun secret du serveur principal.
F15 / P2	Oriane sous adaptateur.	Auth documentée et une recherche réelle validée, avec provenance.
F16 / P2	Autonomie BNB simulée.	Backtest hors échantillon et simulation journalisée avec limites déterministes.
F17 / P3	Exécution BNB réelle sous mandat borné.	Instrument, venue, capital, limites et permissions définis ; rapprochement et arrêt vérifiés.

9. Module BNB : cadrage et architecture de trading

Le dépôt RomThpt/bnb audité contient deux documents. Il décrit des agents vendeurs de services via BNB Agent Studio et un événement du 10 novembre ; il ne fournit ni stratégie ni bot de trading. Son commit de référence est da5e2cf9e6dc8399f357d36ac05df7c89ec61583. [S3]

Périmètre possible	Informations requises et implémentation
A — Trader le jeton BNB sur un exchange	Marché et paire autorisés, pays/compte/venue, spot ou autre produit, clés sans retrait, tailles minimales, tick/step, frais, prix et historique.
B — Swaps sur BNB Chain	Réseau/chain_id, RPC, wallet dédié, tokens/contrats, DEX, approvals, gas, slippage, liquidité, confirmations et réorganisations.
C — Vendre des services d'agent	Agent Studio, identité, endpoints MCP/A2A, paiement X402 et environnement testnet ; ce périmètre correspond au guide fourni.

• Pipeline proposé : données horodatées > contrôle fraîcheur/qualité > caractéristiques et analyse croisée > proposition de stratégie > contrôleur de risque déterministe > simulation > registre d'ordres > rapprochement.

• Sources à croiser : prix/carnet/volumes et frais de la venue, données on-chain si pertinentes, calendrier d'événements et actualités avec provenance. Mesurer leur retard et leur fiabilité. Un événement politique ne constitue pas une causalité de rendement démontrée.

• Backtest : données point-in-time, séparation apprentissage/validation/test, frais, slippage, liquidité et biais de survivance. Suivre rendement net, drawdown, exposition, pertes et stabilité hors échantillon.

• Contrat d'autonomie : compte dédié, instruments permis, capital maximal, exposition par ordre/jour, perte journalière, levier, slippage, fréquence et durée du mandat. Les valeurs restent à fixer avant toute exécution réelle.

• Exécution : worker indépendant du chat, journal durable, identifiant client stable par intention et rapprochement des ordres/trades/positions. Un timeout ou un 5xx crée un état inconnu à réconcilier avant un nouvel envoi.

• L'identifiant client d'un exchange aide la déduplication sans garantir une exécution exactly-once globale. Sur chaîne, suivre nonce, hash, remplacement, receipt et confirmations.

• Le LLM prépare l'analyse et explique la décision. Le composant d'exécution applique les limites, vérifie les paramètres et utilise les permissions dédiées.

• L'ordre vocal « trade BNB » ne suffit pas à définir le mandat. Une demande ambiguë doit être clarifiée et peut rester en simulation. Les données de wallet, seeds et credentials ne transitent jamais dans le prompt.

La configuration Doppler lvx/bnb contient des noms liés au RPC, wallet et administration. Leur présence n'établit ni les droits d'un exchange, ni un moteur de stratégie opérationnel. Aucun wallet, position ou ordre réel n'a été interrogé pendant cette mission.

10. Copier MAVAI dans Lovable sans perdre l’existant

Voie recommandée selon la documentation officielle : remixer le projet Lovable existant dans le workspace cible. Le projet source déclaré est https://lovable.dev/projects/52e102a1-1fed-4dca-9640-6b1406da9cd2. La disponibilité de l'action sur ce compte reste à vérifier dans l'éditeur. [S11]

• Ouvrir Project settings > Project actions > Remix et nommer la copie « MAVAI — DR MANATHAN ». Conserver les instructions du projet.

• Contrôler les éléments copiés : code, structure de base, fonctions et noms des secrets. Reconfigurer valeurs, utilisateurs/données nécessaires, connexions, GitHub et publication selon les besoins de la copie.

• Vérifier les endpoints et URLs codés en dur avant l'exécution du clone. Utiliser des données de démo ou migrer seulement les données explicitement voulues avec leur propriétaire.

• Connecter la copie à GitHub par le flux Lovable : ce flux crée un nouveau dépôt. Réappliquer ensuite les ajouts du livrable et garder la référence de base.

• Appliquer d'abord les étapes P0 du prompt fourni. Tester le socle et chaque petite tranche avant d'ajouter la suivante. Conserver une version de retour arrière.

Alternative officielle : Import project to Lovable > Upload code > archive ZIP (limite publiée de 250 Mo). L'archive livrée exclut secrets, dépendances, builds et historique .git. Cette procédure peut reconstruire l'application et nécessite une comparaison avec la référence ; elle ne transporte pas les utilisateurs ou les valeurs de secrets. La synchronisation

GitHub seule n'importe pas un dépôt existant dans un nouveau projet. [S12, S13]

L'archive proposée contient le code MAVAI et l'exemple SDK ajouté. Elle ne contient pas encore les fonctionnalités DR MANATHAN décrites dans ce cahier. Le prompt Lovable précise l'ordre de leur implémentation. Aucun projet Lovable n'a été remixé ou publié par cette mission.

11. Roadmap, dépendances et estimation

Lot	Livrables et dépendances	Sortie / charge indicative
L0 — Préparer	Copie Lovable, backend propre, secrets/mappings, baseline et retour arrière.	0,5-1 jour ; clone sain et tests de base.
L1 — Démo Open Data	Mode DR, routeur de commandes, accusés, Hémicycle lecture, sources et QR. Dépend de L0 ; voix sous réserve du correctif ElevenLabs.	1-3 jours ; scénario complet vérifiable. Pour aujourd'hui, réduire à une seule verticale.
L2 — Généraliste utile	Documents PDF/CSV, recherche, Featherless, exports et matrice de capacités.	3-5 jours après L1 ; recette docs/sources/isolation.
L3 — Actions durables / médias	Jobs, annulation/réconciliation, budgets, Seedance après déblocage compte, Oriane après doc auth.	3-5 jours ; reprise après reload et pannes maîtrisées.
L4 — BNB simulation	Choix A/B/C, connecteurs de données, stratégie, backtest et paper trading.	1-3 semaines selon périmètre ; résultats hors échantillon et journal revus.
L5 — Autonomie réelle	Mandat chiffré, permissions dédiées, pilote borné et suivi.	Estimation après L4 ; aucune date de mise en service financière garantie.

Charges indicatives pour un développeur connaissant le socle ; elles dépendent des accès, de la disponibilité des APIs et des retours de recette. Les lots BNB peuvent être pilotés séparément. Une interface ressemblant à ChatGPT ne représente pas à elle seule la livraison du lot généraliste.

Démo hackathon suggérée (3 minutes) : QR et entrée sur la copie > « Passe en mode DR MANATHAN » > accusé et badge > question Hémicycle bornée > tableau/graphique avec source et date > « Mode silence » > résultat écrit > retour au module audiovisuel existant. Tant que le TTS reste refusé, présenter explicitement la commande textuelle et le blocage audio.

12. Recette, exploitation et retour arrière

Contrôle	Critère
Socle audiovisuel	Hub /chat, projets, renommage, archivage, mémoire, personnages/versions, musique, audio, frames et thèmes conservés.
Isolation	Deux comptes et deux projets ; refus des IDs tiers pour outils, fichiers, jobs, mémoire et liens de conversations.
Voix	Texte/audio identiques ; accusé avant effet ; arrêt prioritaire ; micro et sortie indépendants ; session expirée et autoplay traités.
Erreurs fournisseur	401/403/429, timeout, failed/canceled/moderated : état honnête, historique conservé et absence de resoumission facturable aveugle.
Données	Sources, période, versions, montants et agrégats vérifiables. Résultat partiel signalé comme partiel.
Secrets	Aucune valeur dans bundle navigateur, SSE, logs, prompt, QR ou archive ; droits du runtime limités aux services nécessaires.
Jobs	Reload et redémarrage sans double effet ; annulation et reprise vérifiées ; coût/usage associés au bon propriétaire.
QR	URL HTTPS de la copie publiée, texte parallèle, contraste/marge et scan téléphone sur projection.
Retour arrière	Flags par capacité, version précédente disponible, migrations additives par Lovable ; désactivation d'un connecteur garde le chat et les médias utilisables.

État des contrôles exécutés ici : bun run test = 79 réussis ; higgsfield:test = 17 réussis ; higgsfield:check et ESLint ciblé = OK ; bun run build = OK, avec avertissements existants de dépréciation/inputValidator et résolution Vite. La commande test native du package sprites2d-core échoue sur un chemin src/test/setup.ts absent. Avec une configuration temporaire isolée, ses 16 tests passent. Ce défaut de configuration est à corriger dans une tranche distincte, sans le masquer.

Le SDK Higgsfield installé est @higgsfield/client 0.2.6. Son polling connaît completed, failed et nsfw. D'autres variantes d'annulation/modération peuvent attendre le timeout ; le validateur de l'exemple les rejette si elles sont renvoyées, et ne déclare jamais un succès sur timeout. Pour le produit, utiliser un suivi durable de requête et la récupération d'état décrits par le fournisseur. [S1, S2]

Exemple livré : examples/higgsfield/index.ts ; commandes bun run higgsfield:check, bun run higgsfield:test et bun run higgsfield:example. Le dernier déclenche une génération payante. HF_CREDENTIALS doit être injecté côté serveur ou saisi dans .env.local ignoré. Une seule tentative a été lancée ici et a été refusée ; aucune relance automatique.

13. Questions de mise en œuvre à résoudre dans les lots concernés

• Choisir le workspace Lovable cible et l'URL finale du clone ; ces valeurs déterminent le QR de production.

• Réconcilier les noms Doppler et ceux attendus par le runtime du clone. Vérifier le secret de chiffrement et les connexions utilisateur sans modifier celles de l'instance originale.

• Débloquer l'accès TTS ElevenLabs et le compte/modèle Higgsfield. Les crédits OpenAI ne règlent pas les crédits Higgsfield.

• Obtenir la documentation d'auth Oriane et préciser le besoin concret : recherche sociale, analyse vidéo ou création d'assets.

• Définir BNB A/B/C, instrument, venue, pays/compte, capital et mandat chiffré avant les lots d'exécution financière.

14. Sources et traçabilité

[S1] SDK officiel Higgsfield — https://docs.higgsfield.ai/docs/how-to/sdk

[S2] Seedance 2.5, référence du modèle — https://console.higgsfield.ai/models/bytedance/seedance-2.5/text-to-video/api-reference

[S3] Dépôt BNB fourni, référence auditée — https://github.com/RomThpt/bnb/tree/da5e2cf9e6dc8399f357d36ac05df7c89ec61583

[S4] Outils OpenAI ; projets ChatGPT — https://developers.openai.com/api/docs/guides/tools

[S4b] Projets ChatGPT — https://help.openai.com/en/articles/10169521-projects-in-chatgpt

[S5] Outils Anthropic — https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview

[S5b] Mémoire API Claude — https://platform.claude.com/docs/en/agents-and-tools/tool-use/memory-tool

[S6] Outils et agents Mistral — https://docs.mistral.ai/studio/agents/agent-tools

[S7] Outils Gemini — https://ai.google.dev/gemini-api/docs/tools

[S8] Hémicycle MCP et outils — https://hemicycle.ai/docs/mcp

[S8b] Outils, disponibilités et coût Hémicycle — https://hemicycle.ai/docs/mcp/outils

[S9] Featherless plan authentifié — https://featherless.ai/docs/api-reference-plan

[S9b] Featherless tool calling — https://featherless.ai/docs/tool-calling

[S10] Oriane outils et documentation — https://www.oriane.xyz/free-tools

[S10b] Oriane OpenAPI public — https://connect.oriane.xyz/rest/docs

[S11] Lovable Remix — https://docs.lovable.dev/features/projects/remix

[S12] Import de projet Lovable — https://docs.lovable.dev/features/projects/import-project

[S13] Intégration GitHub Lovable — https://docs.lovable.dev/integrations/github

[S14] Doppler CLI — https://docs.doppler.com/docs/cli

[S15] Chat Completions OpenAI — https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create

[C1] Règles du dépôt MAVAI — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/AGENTS.md

[C2] Dépendances et scripts — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/package.json

[C3] Chat canonique et prompt — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/src/lib/chat-stream.server.ts#L92

[C4] Voix serveur — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/src/lib/mavai-ai.server.ts#L105

[C5] Fournisseurs et modèles — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/src/lib/user-model.server.ts

[C6] Session voix — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/src/components/voice/voice-session.ts

[C7] Diagnostic — https://github.com/vgactech/MAVAI/blob/d82ccc8091a40cbc57f67cb85a292f21928cbef1/src/lib/chat-health.functions.ts

Vérifications documentaires et techniques du 14 novembre 2026. Les preuves JSON/CSV annexées détaillent les endpoints, dates et limites des tests. Les documents de recherche contiennent une comparaison ciblée des plateformes demandées ; ils ne prétendent pas inventorier tout le marché.